

機械電子創成発展実験・実習

コンピュータ工学研究室

「組み込み開発」編

2025 年度

## 目次

|                      |       |
|----------------------|-------|
| 1. Arduino UNO について  | 3/19  |
| 2. LED の点滅           |       |
| 2-1 ボード内 LED の点滅     | 4/19  |
| 課題 2-1A              | 4/19  |
| 2-2 点滅パターンの繰り返し      | 4/19  |
| 課題 2-2A              | 5/19  |
| 2-3 マイコンボード外 LED の点灯 | 6/19  |
| 課題 2-3A              | 7/19  |
| 2-4 負論理での LED の点灯    | 7/19  |
| 課題 2-4A              | 8/19  |
| 2-5 外部スイッチ入力         | 9/19  |
| 課題 2-5A              | 10/19 |
| 課題 2-5B              | 10/19 |
| 3. DC モータの駆動         |       |
| 3-1 モータドライバの利用       | 11/19 |
| 課題 3-1A              | 13/19 |
| 3-2 モータの回転速度の制御      | 13/19 |
| 課題 3-2A              | 15/19 |
| 課題 3-2B              | 15/19 |
| 4. サーボモータの駆動         |       |
| 4-1 PWM による角度制御      | 16/19 |
| 課題 4-1A              | 17/19 |
| 4-2 ライブラリを用いた角度制御    | 18/19 |
| 課題 4-2A              | 19/19 |
| 課題 4-2A              | 19/19 |

## 1. Arduino UNO について

本実験で用いるマイコンボード Arduino UNO の外観を図 1-1-1 に示す。以下の場所からデータシートダウンロードできる。

<https://docs.arduino.cc/hardware/uno-rev3>

<https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

(プログラミングに関しては <https://www.arduino.cc/reference/en/> を参照のこと)

電源は USB ケーブル経由でパソコンから供給する。パソコンに USB ケーブルで接続せず、独立して使用する場合は AC アダプタ等から DC ジャック経由で供給することも可能である。

ArduinoUNO には多くの入出力ピンが用意されている。代表的なものを以下に示す。

### 1) 電源ピン関係

|      |         |
|------|---------|
| +3V3 | 3.3V 出力 |
| +5V  | 5V 出力   |
| GND  | グランド    |
| GND  | グランド    |
| VIN  | 外部電源入力  |

### 2) アナログ入出力

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| A0～A5 | 疑似アナログ入出力、入力か出力かはプログラムで指定<br>図 1-1 において A0～A5 で指定されているピン<br>プログラムでも A0～A5 で指定す |
|-------|--------------------------------------------------------------------------------|

### 3) デジタル入出力

|      |                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------|
| 0～13 | デジタル入出力ピン、入力か出力かはプログラムで指定<br>図 1-1 において、D0～D13 で指定されているピン<br>プログラムではピン番号 0～13 で指定<br>ただし、0,1,13 ピンは以下の通り特殊用途用。 |
|------|----------------------------------------------------------------------------------------------------------------|

|           |                                 |
|-----------|---------------------------------|
| 0/RX,1/TX | USB 入出力と兼用なので、PC と接続しているときは使用不可 |
| 13        | 内部 LED と兼用。電流制限抵抗内蔵             |

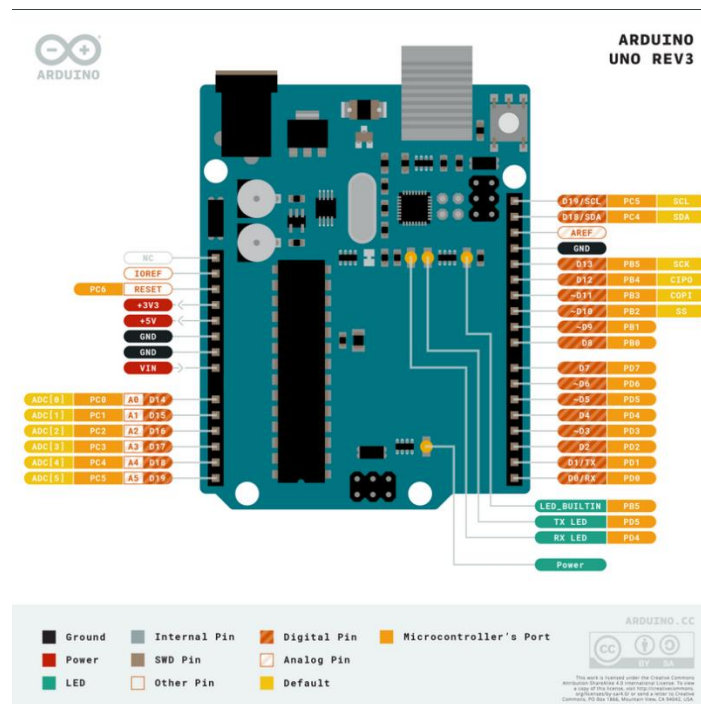


図 1-1-1 Arduino UNO のピン配置

(<https://docs.arduino.cc/hardware/uno-rev3> ⇒ PINOUT)

## 2. LED の点滅

### 2-1 ボード内 LED の点滅

下記 list 2-1 のプログラムでは 0.5 秒ごとに ON と OFF を繰り返すプログラムである。また、`loop()`関数は無限ループとなるため、ひたすら点滅を繰り返す。

```
// list 2-1
// Blink on-board LED for 1 Hz
// 2022.08.26 H.Arai

int LED_PIN = 13; // ボード内 LED ピンの番号をグローバル変数で定義

void setup(){           // 最初に 1 回だけ起動される初期化处理
  pinMode(LED_PIN, OUTPUT); // LED_PIN を出力に設定
}

void loop(){            // 無限に繰り返されるメイン処理
  digitalWrite(LED_PIN, HIGH); // LED_PIN に HIGH(5V)を出力、LED 点灯
  delay(500);             // 500 ms 待機
  digitalWrite(LED_PIN, LOW);  // LED_PIN に LOW(0V)を出力、LED 消灯
  delay(500);             // 500 ms 待機
}
```

list 2-1 ボード内 LED の点滅

### 課題 2-1A

list 2-1 のプログラムにおいて、100 ms 点灯、900 ms 消灯を繰り返す点滅プログラムを作成せよ。

### 2-2 点滅パターンの繰り返し

次の list 2-2 に 3 回点滅のパターンを無限に繰り返すプログラムを示す。このプログラムは、ボード内の LED を以下の図 2-2-1 の通り点滅させる。すなわち、**TimeOn** の時間点灯、**TimeOff** の時間消灯を **RepeatCount** の回数繰り返したのち、**TimeInterval** の時間だけ消灯させる。

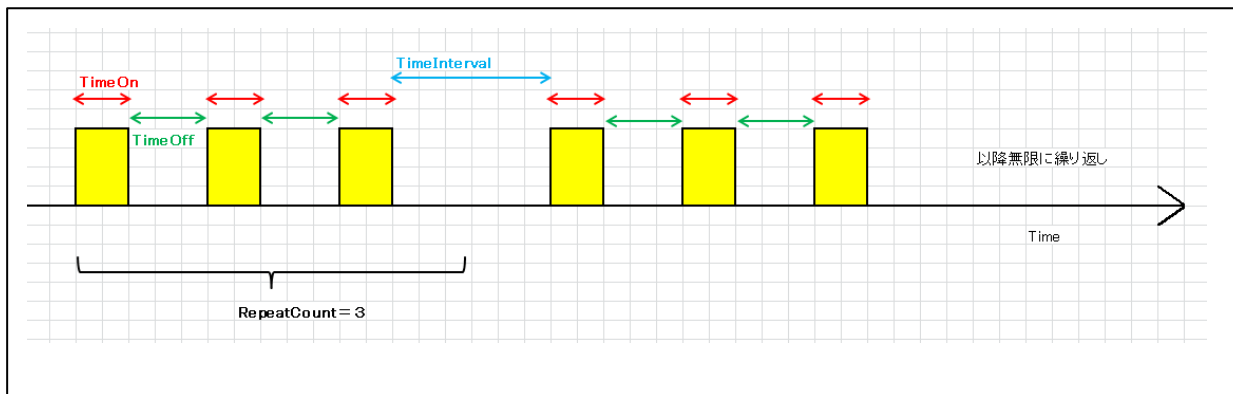


図 2-2-1 3 回点滅を繰り返す

```
// list2-2
// Blink on-board LED for a pattern
// 2022.08.26 H.Arai

int LED_PIN = 13;
int TimeOn = 100;      // 点灯している時間(ms)
int TimeOff = 200;     // 消灯している時間(ms)
int BlinkCount = 3;    // 点滅回数
int TimeInterval = 1000; // 所定回数点滅した後のインターバル時間(ms)
                        // TimeInterval > TimeOff is Required

void setup(){
  pinMode(LED_PIN, OUTPUT);
}

void loop(){
  a_pattern();
  delay(TimeInterval);
}

void a_pattern(){
  for(int i=0 ; i<BlinkCount; i++){
    digitalWrite(LED_PIN, HIGH);
    delay(TimeOn);
    digitalWrite(LED_PIN, LOW);
    if (i<BlinkCount-1) // don't wait after last blink
      delay(TimeOff);
  }
}
```

list 2-2 ボード内 LED で所定の点滅を繰り返す

## 課題 2-2A

モールス信号の SOS (・・・ ——— ……) を無限に点滅させるプログラムを作成せよ。なお短音を 1 単位時間としたときに、長音は 3、音間は 1、文字間は 3、単語間は 7 単位時間と規定されている。また、1 単位時間を何秒とするかは任意であるので、これを初期値で設定できるようにせよ。また、"S"と"O"の点滅はそれぞれ blinkS0 と blinkO0 という関数にすること。

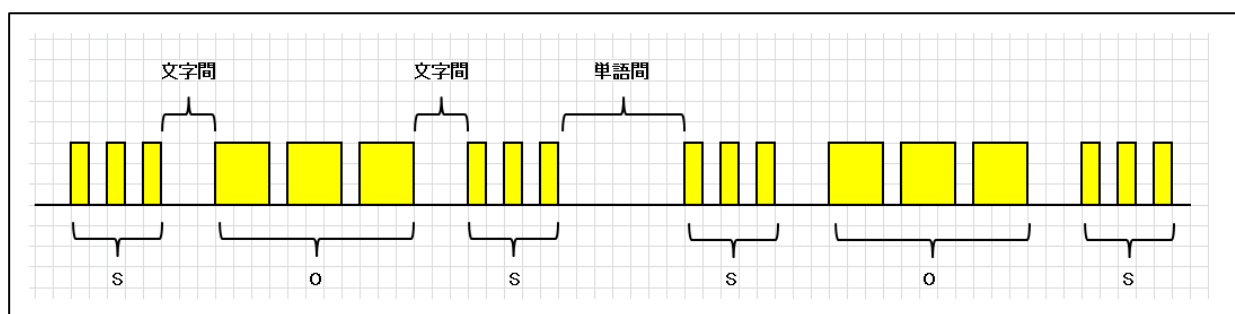


図 2-2-2 SOS の点滅パターン

参考 : ITU 規格

[https://www.itu.int/dms\\_pubrec/itu-r/rec/m/R-REC-M.1677-1-200910-I!!PDF-E.pdf](https://www.itu.int/dms_pubrec/itu-r/rec/m/R-REC-M.1677-1-200910-I!!PDF-E.pdf)

2 Spacing and length of the signals

2.1 A dash is equal to three dots.

2.2 The space between the signals forming the same letter is equal to one dot.

2.3 The space between two letters is equal to three dots.

2.4 The space between two words is equal to seven dots.

## 2-3 マイコンボード外 LED の点灯

マイコンボード外に LED を接続して点滅させるための回路図を図 2-3-1 に示す。この回路ではデジタルの 10 ピンを出力として使い、ここに HIGH を出力することにより LED が点灯し、LOW を出力することにより LED が消灯する回路となる。すなわち、正論理で LED を点滅させることになる。この回路で list2-1 と同様に点滅させるプログラムを list2-3 に示す。

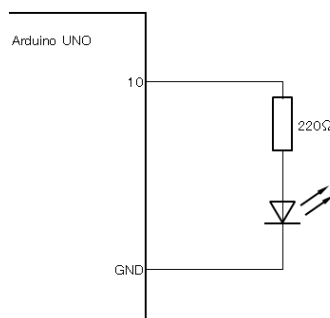


図 2-3-1 外部 LED の接続

```

// list 2-3
// Blink External LED for 1 Hz
// 2022.08.26 H.Arai

int LED_PIN = 10;                                // ボード内 LED ピンの番号をグローバル変数で定義

void setup(){                                    // 最初に 1 回だけ起動される初期化处理
  pinMode(LED_PIN, OUTPUT);                      // LED_PIN を出力に設定
}

void loop(){                                     // 無限に繰り返されるメイン処理
  digitalWrite(LED_PIN, HIGH);                  // LED_PIN に HIGH(5V)を出力、LED 点灯
  delay(500);                                   // 500 ms 待機
  digitalWrite(LED_PIN, LOW);                   // LED_PIN に LOW(0V)を出力、LED 消灯
  delay(500);                                   // 500 ms 待機
}

```

list 2-3 外部 LED の点滅

## 課題 2-3A

ブレッドボード上に図 2-3-1 の回路を作成し、課題 2-2A の SOS の点滅プログラムの出力ピン番号を変更した上で実行せよ。

## 2-4 負論理での LED の点灯

図 2-4-1 は、LED1 を正論理、LED2 を負論理で点灯するように接続した回路図である。すなわち、LED1 は 10 ピンに HIGH を出力したときに点灯、LOW を出力したときに消灯するが、LED2 は 9 ピンに HIGH を出力したときに消灯、LOW を出力したときに点灯する回路となる。この回路で LED1 と LED2 を交互に点灯させるプログラムを list 2-4 に示す。

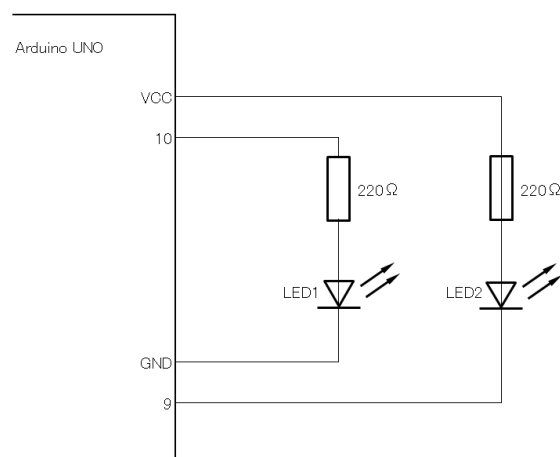


図 2-4-1 正論理と負論理の外部 LED の接続

```

// list2-4
// Blink external two leds
// led1 is positive-logic, led2 is negative logic
// 2022.08.26 H.Arai

int LED_PIN1 = 10;
int LED_PIN2 = 9;
int Time1 = 100;           // LED1 が点灯している時間(ms)
int Time2 = 200;           // LED2 が点灯している時間(ms)

void setup(){
  pinMode(LED_PIN1, OUTPUT);
  pinMode(LED_PIN2, OUTPUT);
}

void loop(){
  digitalWrite(LED_PIN1, HIGH);    // LED1 ON (Positive Logic)
  digitalWrite(LED_PIN2, HIGH);    // LED2 OFF (Negative Logic)
  delay(Time1);
  digitalWrite(LED_PIN1, LOW);     // LED1 OFF (Positive Logic)
  digitalWrite(LED_PIN2, LOW);     // LED2 ON (Negative Logic)
  delay(Time2);
}

```

list 2-4 正論理と負論理の外部 LED を交互に点灯

## 課題 2-4A

ブレッドボード上に図 2-4-1 の回路を作成し、LED1 と LED2 が図 2-4-1 の通り点滅するプログラムを作成せよ。

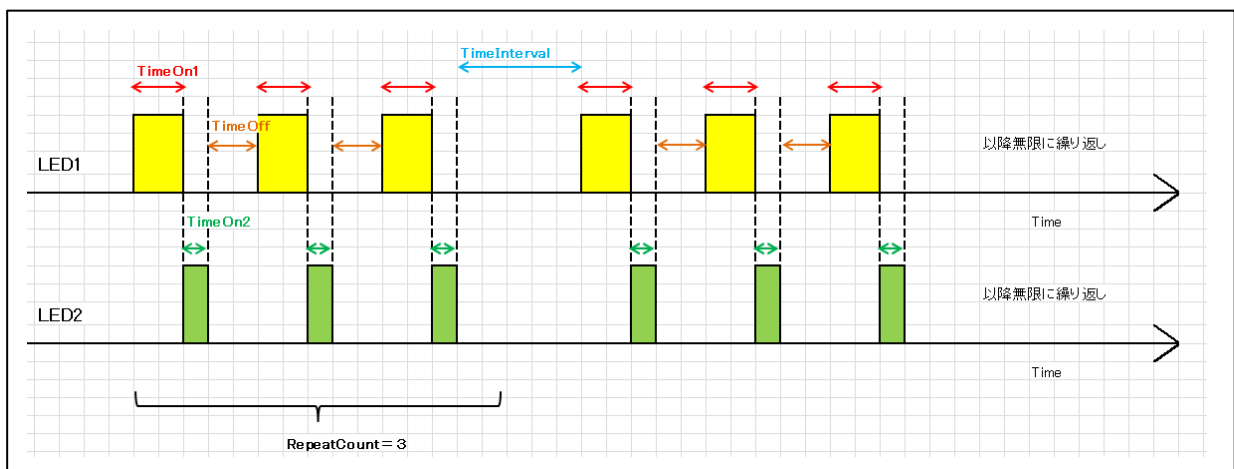


図 2-4-1 2つの LED が 3 回点滅を繰り返す



## 2-5 外部スイッチ入力

外部スイッチ入力を用いる場合の回路の例を図 2-5-1 に示す。この回路は負論理であり、スイッチが OFF の時に 11 ピンに HIGH が、スイッチが ON の時に 11 ピンに LOW が入力される。この回路を用いて、スイッチが押されている間のみ LED を点灯するプログラムを list2-5 に示す。

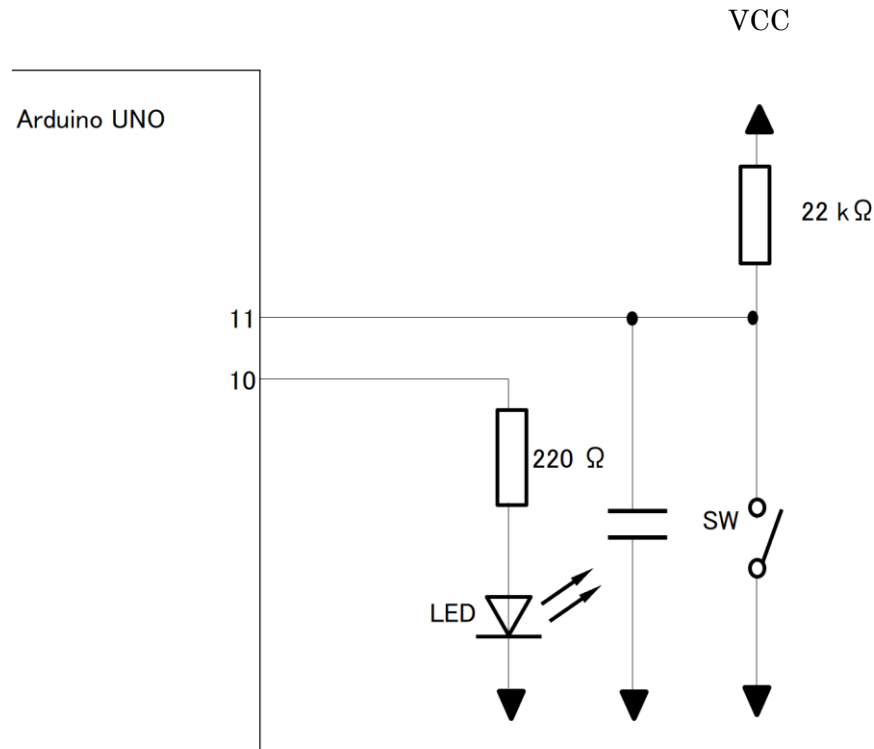


図 2-5-1 外部スイッチ入力（負論理）

```
// list 2-5
// Light LED according to switch input
// 2022.08.26 H.Arai

int LED_PIN = 10;                // LED ピンの番号を定義
int SW_PIN = 11;                // SW ピンの番号を定義

void setup(){
  pinMode(LED_PIN, OUTPUT);      // LED_PIN を出力に設定
  pinMode(SW_PIN, INPUT);       // SW_PIN を入力に設定
}

void loop(){
  if (digitalRead(SW_PIN)==HIGH) // SW が OFF の時
    digitalWrite(LED_PIN, LOW); // LED を消灯
  else                          // SW_PIN==LOW、すなわち SW が ON の時
    digitalWrite(LED_PIN, HIGH); // LED を点灯
```

list2-5 スwitchが押されている間のみ LED を点灯

## 課題 2-5A

ブレッドボード上に図 2-5-1 の回路を作成し、スイッチが押されるたびに LED の点灯と消灯を切り替えるプログラムを作成せよ。なお、チャタリング対策はおこなわなくて良いものとする。

## 課題 2-5B

SW1 と SW2 の 2 つのスイッチを用い、LED の点滅周波数を 1 Hz から 100 Hz の間で変えられるプログラムを作成せよ。初期状態は 1 Hz で点滅しており、SW1 を押している間は徐々に点滅周波数が高くなる。最高で 100Hz になった状態でそれ以上は早くならないものとする。また SW2 を押している間は徐々に点滅周波数が低くなり、裁定で 1Hz になった状態でそれ以上は遅くならないものとする。なお、点滅のデューティ比は 50% で一定とする。

### 3. DC モータの駆動

#### 3-1 モータドライバの利用

Arduino は GPIO に 5V を出力するか 0V を出力するかをプログラムで指定できる。理論的にはこの GPIO に DC モータを接続すれば回転と停止をプログラムで制御できることになる。しかしながら、Arduino の GPIO に流せる電流は入力・出力ともに 40 mA である。

<http://www.musashinodenpa.com/arduino/ref/index.php?f=0&pos=1643>

このため、Arduino の出力ピンに DC モータを直結して回転させると 40 mA 以上の電流が流れて Arduino が壊れてしまう可能性が高い。トランジスタを用いて電流増幅させる方法もあるが、Hブリッジ等を組まないと正転と逆転を制御することはできない。一方機械電子創成基礎実験・実習で用いたようにモータドライバ IC を用いることも可能である。しかしながら Arduino には DC モータを駆動するためのモータシールドが存在しており簡単に DC モータを駆動することができるのでこれを用いる。また、モータシールドを用いることにより正転と逆転の制御なども用意になる。

モータシールドは Arduino の基板に直接載せるように設計されており、自動的に Arduino の所定の GPIO とモータシールドの所定のピンが接続される。図 3-1-1 の回路図が提供されているが、これは誤りであり、Arduino のピンの対応は表 3-1-1 のようになっている。

[https://wiki.seeedstudio.com/Motor\\_Shield\\_V2.0/](https://wiki.seeedstudio.com/Motor_Shield_V2.0/)

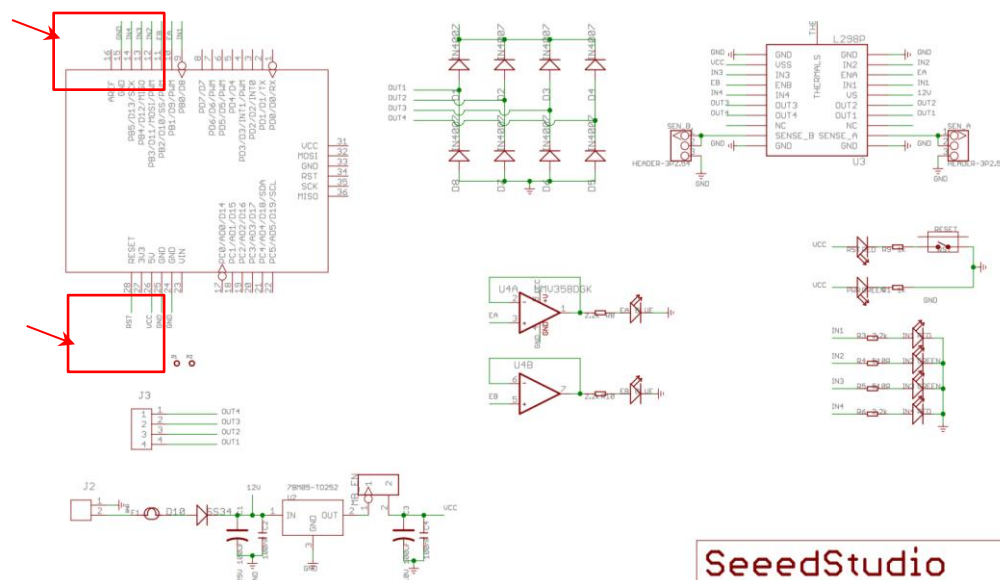


図 3-1-1 モータシールドの回路図

表 3-1-1 モータシールドと Arduino のピンの関係

| Arduino のピン番号 | モータシールドの信号名 | モータシールドの信号の意味  |
|---------------|-------------|----------------|
| 8             | IN1         | モータ 1 の制御信号 1  |
| 9             | EA          | モータ 1 のイネーブル信号 |
| 10            | EB          | モータ 2 のイネーブル信号 |
| 11            | IN2         | モータ 1 の制御信号 2  |
| 12            | IN3         | モータ 2 の制御信号 1  |
| 13            | IN4         | モータ 2 の制御信号 2  |
| 14            | GND         | グラウンド          |
| GND           | GND         | グラウンド          |
| GND           | GND         | グラウンド          |
| 5V            | VCC         | 電源             |
| RESET         | RST         | リセット           |

list 3-1 にモータの正転と逆転を繰り返すプログラムを示す。また、このプログラムに関係する回路の接続を図 3-1-2 に示す。

```
// list 3-1
// DC Motor Control(Simple Version)
// repeat {rotate CW , stop , rotate CCW, stop}
//   CW : Clock Wise, CCW:Counter Clock Wise
// 2022.08.31 H.Arai

int MOTOR_EN = 9;           // Motor1 Enable pin
int MOTOR_IN1= 8;           // Motor1 Control pin1
int MOTOR_IN2= 11;          // Motor1 Control pin2
int INTERVAL_ON = 1000;     // rotate on interval 1000 ms
int INTERVAL_STOP = 500;    // rotate off interval 500 ms
void setup(){
  pinMode(MOTOR_EN, OUTPUT);
  pinMode(MOTOR_IN1, OUTPUT);
  pinMode(MOTOR_IN2, OUTPUT);
}

void loop(){
  MotorCW();                 // rotate CW
  delay(INTERVAL_ON);
  MotorStop();               // stop motor
  delay(INTERVAL_STOP);
  MotorCCW();                // rotate CCW
  delay(INTERVAL_ON);
  MotorStop();               // stop motor
  delay(INTERVAL_STOP);
}

void MotorCW(){
  digitalWrite(MOTOR_EN,HIGH);
  digitalWrite(MOTOR_IN1,HIGH);
  digitalWrite(MOTOR_IN2,LOW);
}

void MotorCCW(){
  digitalWrite(MOTOR_EN,HIGH);
  digitalWrite(MOTOR_IN1,LOW);
  digitalWrite(MOTOR_IN2,HIGH);
}

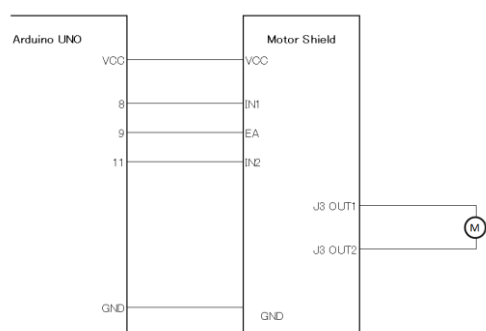
void MotorStop(){
  digitalWrite(MOTOR_EN,LOW);
}
```

list 3-1 モータシールドを用いた DC モータの制御

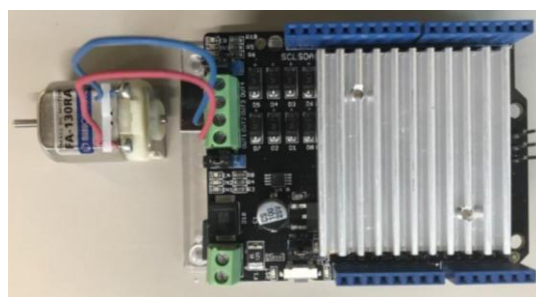
本モータシールドは 2 つの DC モータを制御できる。list 3-1 と図 3-1-2 はモータ 1 を制御する場合の接続である。

補足 1：今回はモータシールドへの電源供給は Arduino からおこなっており、すなわち USB ケーブルで PC から供給していることになる。しかしながら USB ケーブルでの電流供給能力は 500 mA までであり、それ以上の電流を供給すると PC の USB ポートが破損する可能性がある。このため、大容量の電流を必要とするモータを駆動する場合には、モータシールドへの電源供給を別途用意する必要がある。

補足 2：モータシールドを Arduino に載せることによって自動的に表 3-1-1 の接続となるが、あえてモータシールドを Arduino に載せずに自分でジャンパー線で接続すれば、表 3-1-1 以外の Arduino のピンでモータを制御することも可能である。



(a) 回路図



(b) 配線図

図 3-1-2 Motor Shield の接続

### 課題 3-1A

スイッチ 1 を押している間だけモータが正回転し、スイッチ 2 を押している間はモータが逆回転するプログラムを作成せよ。なお、両方のスイッチが押されているときはモータは停止させておくものとする。（Arduino をモータシールドに挿した状態の時は表 3-1-1 のデジタルピンはスイッチ入力等には使えないので、スイッチにはデジタルピンの 2～7 のいずれかを使用すること。）

### 3-2 モータの回転速度の制御

Enable ピン(EA,EB)を PWM で制御することにより、モータの回転速度を制御することが可能である。list3-2 では、正転の信号をモータシールドの IN1 と IN2 に与えたうえで、EA に PWM 信号を与えることによってモータの回転速度を段々早くしている。具体的には、 $n/255$  の PWM を生成しており、 $n$ （プログラム中の Speed）を 1～255 に変化させて、 $1/255 \sim 255/255$  のパルス幅の PWM 信号を EA に入力している。

なお、Arduino には PWM 信号を生成することにより疑似アナログ電圧を出力する機能があり、ピン名に「~」がついているピンであればデジタルピンでも使用できる。

<https://cdn.arduino.cc/reference/jp/language/functions/analog-io/analogwrite/>

この機能を用いて list3-2 と同じような動きをさせるプログラムの例を list3-2analogWrite に示す。なお、list3-2analogWrite の loop()以外の処理は list3-2 と同じである。

```

// list 3-2
// DC Motor Control(Speed Control)
// 2022.08.31 H.Arai

int MOTOR_EN = 9;           // Motor1 Enable pin
int MOTOR_IN1= 8;           // Motor1 Control pin1
int MOTOR_IN2= 11;          // Motor1 Control pin2

void setup(){
  pinMode(MOTOR_EN, OUTPUT);
  pinMode(MOTOR_IN1, OUTPUT);
  pinMode(MOTOR_IN2, OUTPUT);
}

void loop(){
  MotorCW();                 // rotate CW
  for (int Speed=0; Speed<=255; Speed=Speed+10){
    digitalWrite(MOTOR_EN,HIGH);
    delay(Speed);
    digitalWrite(MOTOR_EN,LOW);
    delay(255-Speed);
  }
  MotorStop();
  delay(1000);
}

void MotorCW(){
  digitalWrite(MOTOR_IN1,HIGH);
  digitalWrite(MOTOR_IN2,LOW);
}

void MotorStop(){
  digitalWrite(MOTOR_EN,LOW);
}

```

list 3-2 PWM による DC モータの回転速度の制御

```

void loop(){
  MotorCW();                               // rotate CW
  for (int Speed=0; Speed<=255; Speed++){
    analogWrite(MOTOR_EN, Speed);
    delay(30);                             // keep desired speed for 30ms
  }
  MotorStop();
  delay(1000);
}

```

list 3-2 analogWrite()を用いた PWM の生成

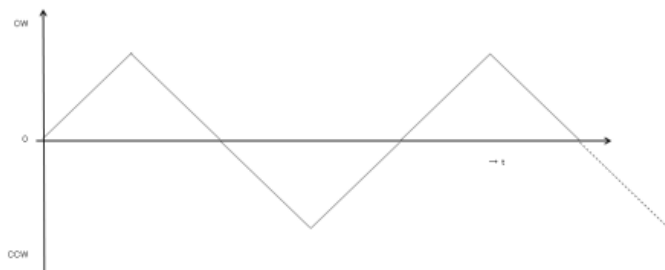
### 課題 3-2A

停止(PWM のデューティ比 0%)から最大速度(PWM のデューティ比 100%)の間で、スイッチ 1 を押している間はモータの回転速度が増大し、スイッチ 2 を押している間はモータの回転速度が減少するプログラムを作成せよ。

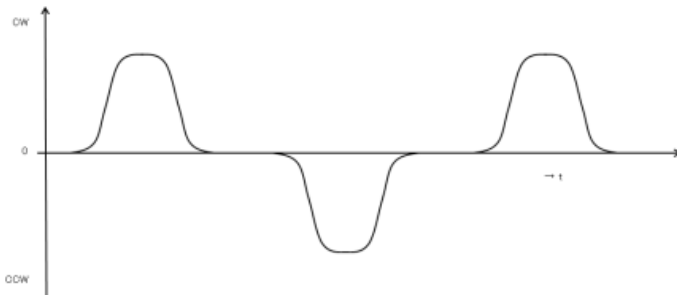
なお、analogWrite()の第 2 引数で指定できる値の範囲は 0～255 の間であり、0 の時デューティ比 0%、255 の時デューティ比 100%となる。

### 課題 3-2B

モータの回転速度が徐々に早くなったのち、徐々に遅くなって逆回転で速くなっていき、さらにまた速度が遅くなって正回転となることを繰り返すようなプログラムを作成せよ。



ただし、PWM のデューティ比と回転速度は必ずしも比例しないことと、デューティ比が一定の値以下の時はトルク不足で回転しないことは許容する。すなわち、実際には体感的に以下のような回転速度になる。



## 4. サーボモータの駆動

### 4-1 PWM によるサーボモータの角度制御

サーボモータを制御するための制御信号は通常の 5V のデジタル信号の PWM なので、特段のドライバ等には必要ではなく、Arduino と直結するだけで良い。ただしサーボモータの電源は Arduino から供給できる電流を越えるので、別途用意する必要がある。例えば単 3 電池 3 本で構成された約 4.5 V の電源を接続する場合の接続図を図 4-1-1 に示す。また、サーボモータのコネクタは通常 VCC、GND、信号線の 3 本である。図 4-1-2 に今回用いるサーボモータのコネクタを示す。赤色が VCC、茶色が GND、黄色が信号線である。サーボモータのメーカーによって色が異なるので注意が必要である。

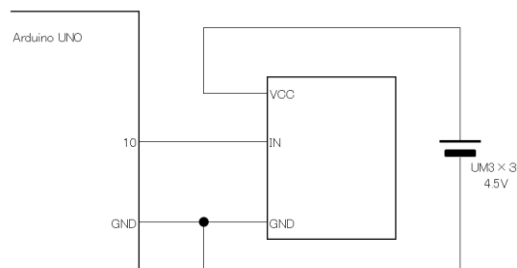


図 4-1-1 サーボの接続回路図



図 4-1-2 サーボのコネクタ

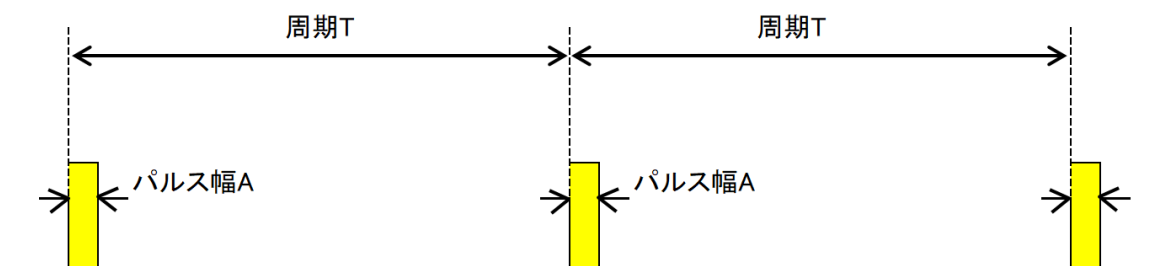


図 4-1-3 PWM (Pulse Width Modulation)

サーボモータの制御には図 4-1-3 に示す PWM、すなわちパルス幅変調を用いる。一般的に周波数  $f$  は 50Hz、すなわち周期  $T$  は 20ms で固定であり、パルス幅  $A$  を変化させることによってサーボモータの角度（時計回り）が変わる。 $A$  は 500 $\mu$ S ~ 2,400 $\mu$ S の間で変化させる。（これは、周波数一定のままデューティ比を 2.5% ~ 12.0% の間で変化させることに相当する。）

交互に 2 つの角度をとるプログラムの例を list 4-1 に示す。このプログラムは上図 4-1-3 のパルス幅  $A$  を、1ms (1,000 $\mu$ S) と 2ms (2,000 $\mu$ S) で交互に切り替えている。20ms のパルスを 100 回出力することに角度を変えているので、各角度を 2 秒間ずつ保持することになる。

$A$  が最小(500 $\mu$ S)の時の角度を  $\theta_{min}$  度、最大(2,400 $\mu$ S)の時の角度を  $\theta_{max}$  度とすると、あるパルス幅  $A$  の時の角度  $\theta$ [度]は以下の通りとなる。

$$\theta = \theta_{min} + (\theta_{max} - \theta_{min}) \times \frac{A - 500}{2,400 - 500}$$

今回のサーボモータでは  $\theta_{min}=0$ 、 $\theta_{max}=180$  なので、list 4-1 のプログラムは以下の 2 つの角度をとることになる。

$$\theta_1 = \theta_{min} + (\theta_{max} - \theta_{min}) \times \frac{1000 - 500}{2,400 - 500} \simeq 45^\circ$$



$$\theta_2 = \theta_{min} + (\theta_{max} - \theta_{min}) \times \frac{2000 - 500}{2,400 - 500} \approx 135^\circ$$

```
// list 4-1
// Servo Motor Control (Two angle)
// 2022.09.1 H.Arai

int SERVO_PIN = 10;           // Motor1 Enable pin
int T = 20;                   // corresponds to 20 ms = 20,000 us
int INTERVAL1 = 1;           // corresponds to 1,000 us
int INTERVAL2 = 2;           // corresponds to 2,000 us

void setup(){
  pinMode(SERVO_PIN, OUTPUT);
}

void loop(){
  for (int i=0; i<100; i++){
    digitalWrite(SERVO_PIN, HIGH);
    delay(INTERVAL1);          // keep HIGH for INTERVAL1 ms
    digitalWrite(SERVO_PIN, LOW);
    delay(T - INTERVAL1);      // keep LOW for T-INTERVAL1 ms
  }
  for (int i=0; i<100; i++){
    digitalWrite(SERVO_PIN, HIGH);
    delay(INTERVAL2);          // keep HIGH for INTERVAL2 ms
    digitalWrite(SERVO_PIN, LOW);
    delay(T - INTERVAL2);      // keep LOW for T-INTERVAL2 ms
  }
}
```

list 4-1 2つの角度を交互に繰り返すサーボ制御プログラム

なお、Arduino では `delay()` は ms 単位の時間を指定するが、`delayMicroseconds()` を用いることにより  $\mu\text{s}$  単位でパルス幅 A を変えることが可能である。但し、`delayMicroseconds()` に渡す引数は 16383 以下でなければならない。すなわち、16ms 以上(16,000 $\mu\text{s}$  以上)の時間を指定することはできない。

パルス周期  $T=20\text{ ms}$  に対して、17 ms は常時 LOW を出力するものとして、3 ms の間のパルス幅を 1,000  $\mu\text{s}$  と 2,000 $\mu\text{s}$  の 2 通りで交互に変更するプログラムを list4-1A に示す。

#### 課題 4-1A

サーボモータの角度を `theta[度]` で `period[s]` 維持する関数 `servo_set(int theta, int period)` を作成し、これ呼び出すことによってサーボモータの角度を、0 度(2 秒)⇒ 90 度(1 秒) ⇒180 度(2 秒)⇒90 度(1 秒)⇒0 度(2 秒)・・・と変化させるプログラムを作成せよ。



```

// list 4-1A
// Servo Motor Control (Two angle)
// 2022.09.1 H.Arai

int SERVO_PIN = 10;           // Motor1 Enable pin
int BASE_T = 17;              // corresponds to 17 ms = 17,000 us
int VARIABLE_T = 3000;        // corresponds to 3 ms = 3,000 us
int INTERVAL1 = 1000;         // corresponds to 1,000 us
int INTERVAL2 = 2000;         // corresponds to 2,000 us

void setup(){
  pinMode(SERVO_PIN, OUTPUT);
}

void loop(){
  for (int i=0; i<100; i++){
    digitalWrite(SERVO_PIN, HIGH);
    delayMicroseconds(INTERVAL1);
    digitalWrite(SERVO_PIN, LOW);
    delayMicroseconds((VARIABLE_T-INTERVAL1));
    delay(BASE_T);
  }
  for (int i=0; i<100; i++){
    digitalWrite(SERVO_PIN, HIGH);
    delayMicroseconds(INTERVAL2);
    digitalWrite(SERVO_PIN, LOW);
    delayMicroseconds((VARIABLE_T-INTERVAL2));
    delay(BASE_T);
  }
}

```

list 4-1A 0度と90度を2秒ごとに繰り返すサーボ制御プログラム

## 4-2 サーボモータライブラリを用いたサーボモータの角度制御

list4-1のようにPWMをdelay関数で実現すると、サーボモータを動かしている最中に別の処理、例えばLEDの1秒ごとに点滅させること等は難しい。サーボモータ制御のためのPWMを割込み処理で自動的に生成してくれるライブラリを用いたプログラムの例をlist4-2に示す。

このプログラムではlist4-1Aと同様、0度と180度を2秒ごとに繰り返している。

```
// list 4-2
// Servo Motor Control (with library)
// 2022.09.1 H.Arai

#include <Servo.h>

Servo myservo;

void setup(){
  myservo.attach(10);
}

void loop(){
  myservo.write(0);
  delay(2000);
  myservo.write(180);
  delay(2000);
}
```

#### 課題 4-2A

サーボの初期角度 0 度とし、角度が 0 度⇒180 度⇒0 度の間でゆっくり首振りをするプログラムを作成せよ。(首振り 1 往復で 5 秒かかる程度)

#### 課題 4-2B

サーボの初期角度 0 度とし、スイッチを押している間だけ角度が 0 度⇒180 度⇒0 度の間でゆっくり首振りをするプログラムを作成せよ。(首振り 1 往復で 5 秒かかる程度)  
なお、スイッチを離した瞬間にその場所で停止するようにすること。